Programming Fundamentals

Credit Hours: 3-1

Course Outline

Introduction to Programming Concepts, Algorithms and Flowcharts, Basics of Computer Systems and Programming Languages, Data Types and Variables, Input and Output Operations, Operators and Expressions, Control Structures (Selection and Iteration), Functions and Parameter Passing, Arrays (One-dimensional and Multi-dimensional), Strings and String Handling, Pointers and Memory Management, Structures and Unions, File Handling, Introduction to Object-Oriented Programming Concepts, Debugging and Error Handling, Basic Problem Solving and Program Design Techniques

16-Week Course Plan

Week 1

Error: Request timed out.

Week 2

Error: Request timed out.

Week 3

Error: Request timed out.

Week 4

Error: Request timed out.

Week 5

Error: Request timed out.

Week 6

Week 6: Arrays (One-dimensional and Multi-dimensional)

Lecture Title: Introduction to Arrays

Learning Objectives:

• Define and declare arrays in a program.

• Understand the concept of one-dimensional and multi-dimensional arrays.

• Learn how to access and manipulate array elements.

Concepts:

Arrays are a fundamental data structure in programming that store multiple values of the same data type in a single variable. A one-dimensional array is a list of elements that can be accessed using a single index. A multi-dimensional array is a table of elements that can be accessed using multiple indices. In C, arrays are declared using the square brackets `[]` after the data type and the name of the array. For example, `int scores[5];` declares a one-dimensional array of five integers. Multi-dimensional arrays are declared using multiple sets of square brackets, for example, `int scores[3][4];` declares a two-dimensional array of three rows and four columns.

Lab: Array Operations

• Create a program that declares and initializes a one-dimensional array of five integers.

• Write a program that declares and initializes a two-dimensional array of three rows and four columns.

• Write a program that demonstrates how to access and manipulate array elements.

Time Distribution: 3-1

Week 7

Week 7: Functions and Parameter Passing (3-1)

Lecture Title: Functions and Parameter Passing

Learning Objectives

* Understand the concept of functions and their importance in programming

* Learn how to declare and use functions in a program

* Understand the different types of parameter passing (pass-by-value and pass-by-reference)

Concepts

A function is a block of code that performs a specific task and can be called multiple times from different parts of a program. Functions are useful for organizing code, reducing repetition, and improving program modularity.

To declare a function, we use the `function` keyword followed by the function name and parameters in parentheses. The function body is enclosed within curly brackets.

```c

return_type function_name(parameter_list) {

// function body

}

```

Functions can take arguments, which are passed to the function using the `call` or `invoke` operator. There are two types of parameter passing: pass-by-value and pass-by-reference.

Pass-by-value: The actual value of the argument is passed to the function, and any changes made to the argument within the function do not affect the original value.

Pass-by-reference: A reference to the actual value of the argument is passed to the function, and any changes made to the argument within the function affect the original value.

```c

void swap(int x, int y) {

int temp = x;

x = y;

y = temp;

}

int main() {

int a = 5;
```

```
int b = 10;

swap(a, b);

return 0;

}
```

In this example, the `swap` function takes two `int` arguments by value. The function swaps the values of `x` and `y` and returns nothing.

Lab

Implement a function that calculates the area and perimeter of a rectangle. The function should take the length and width of the rectangle as arguments and return the area and perimeter as a struct.

Week 8

Error: Connection error.

Week 9

Error: Request timed out.

Week 10

Week 10: Introduction to Object-Oriented Programming Concepts (3-0)

Lecture Title: Object-Oriented Programming Fundamentals

Learning Objectives

• Define object-oriented programming (OOP) and its key concepts

• Identify the benefits of using OOP in programming

• Explain the differences between OOP and procedural programming

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. OOP emphasizes modularity, reusability, and code organization. Key OOP concepts include encapsulation, inheritance, polymorphism, and abstraction. OOP provides a more efficient and maintainable way of designing and developing software systems.

Key Terms:

• **Encapsulation:** The concept of bundling data and its associated methods within a single unit.

• **Inheritance:** The mechanism by which one class can inherit properties and behavior from another class.

• **Polymorphism:** The ability of an object to take on multiple forms.

• **Abstraction:** The process of representing complex systems in a simplified manner.

Benefits of OOP:

• **Modularity:** Easier code organization and maintenance

• **Reusability:** Code can be reused across multiple projects

• **Easier Debugging:** Simpler debugging due to modular code structure

Procedural Programming vs. Object-Oriented Programming:

• **Procedural Programming:** Focuses on procedures and functions

• **Object-Oriented Programming:** Focuses on objects and classes

Week 11

Error: Request timed out.

Week 12

Week 12: Introduction to Object-Oriented Programming Concepts (3-0)

Lecture Title: Object-Oriented Programming Fundamentals

Learning Objectives:

• Define object-oriented programming (OOP) concepts.

• Explain the importance of OOP in software development.

• Identify key OOP principles.

Concepts:

Object-oriented programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. OOP focuses on simulating real-world objects and their interactions. The main principles of OOP include encapsulation, inheritance, and polymorphism.

Encapsulation: Hiding internal details and exposing only necessary information to the outside world.

Inheritance: Creating a new class based on an existing class, inheriting its properties and behavior.

Polymorphism: Ability of an object to take on multiple forms, depending on the context.

OOP provides several benefits, including:

• Improved code modularity and reusability.

• Enhanced code maintainability and flexibility.

• Better support for complex software systems.

Key Terms: Object-Oriented Programming (OOP), Encapsulation, Inheritance, Polymorphism.

References: [List relevant textbooks or online resources]

Week 13

Error: Error code: 402 - {'error': {'message': 'This request requires more credits, or fewer max_tokens. You requested up to 3000 tokens, but can only afford 2232. To increase, visit https://openrouter.ai/settings/credits and upgrade to a paid account', 'code': 402, 'metadata': {'provider_name': None}}, 'user_id': 'user_36ss9pZWJfqzD9zDa5OgHHvP5eS'}

Week 14

Error: Error code: 402 - {'error': {'message': 'This request requires more credits, or fewer max_tokens. You requested up to 3000 tokens, but can only afford 2232. To increase, visit https://openrouter.ai/settings/credits and upgrade to a paid account', 'code': 402, 'metadata': {'provider_name': None}}, 'user_id': 'user_36ss9pZWJfqzD9zDa5OgHHvP5eS'}

Week 15

Error: Request timed out.

Week 16

Week 16: Introduction to Object-Oriented Programming Concepts

3-1

Lecture 1: Object-Oriented Programming (OOP) Concepts

Learning Objectives

* Understand the basic principles of Object-Oriented Programming

* Learn about classes, objects, inheritance, and polymorphism

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. A class is a blueprint or a template that defines the properties and behaviors of an object. An object is an instance of a class, having its own set of attributes (data) and methods (functions). OOP provides several key concepts, including:

* **Encapsulation**: Hiding the implementation details of an object from the outside world

* **Inheritance**: Creating a new class based on an existing class

* **Polymorphism**: The ability of an object to take on multiple forms

* **Abstraction**: Focusing on essential features while ignoring non-essential details

Lecture 2: Classes and Objects

Learning Objectives

* Understand how to define and use classes and objects

* Learn about object properties and methods

Concepts

In OOP, a class is defined using a class definition statement. The class contains properties (data) and methods (functions). An object is created from a class using the `new` keyword. The object has its own set of attributes and methods.

```java
// Example class definition
public class Person {
private String name;
private int age;
public Person(String name, int age) {
this.name = name;
this.age = age;
}
public void displayInfo() {
System.out.println("Name: " + name + ", Age: " + age);
}
}
// Example object creation
Person person = new Person("John Doe", 30);
person.displayInfo();
```

Lecture 3: Inheritance and Polymorphism

Learning Objectives

* Understand how to use inheritance and polymorphism
* Learn about method overriding and method overloading

Concepts

Inheritance allows a new class to inherit the properties and methods of an existing class. Polymorphism enables an object to take on multiple forms.

```java
// Example inheritance
public class Animal {
public void sound() {
System.out.println("The animal makes a sound");
```

```
}
}
public class Dog extends Animal {
public void sound() {
System.out.println("The dog barks");
}
}
// Example polymorphism
Animal animal = new Dog();
animal.sound();
```

Lab 1: Object-Oriented Programming Exercises

* Complete the following exercises:

+ Define a class for a Rectangle and calculate its area and perimeter

+ Create a class for a Circle and calculate its area and circumference

+ Use inheritance to create a class for a Square that extends the Rectangle class

+ Use polymorphism to create a method that can calculate the area of different shapes